大模型辅助的面向对象软件分析、设计与开发 · 阶段 1 课堂研讨

Mandol融合索引增强

约束下推式组合检索

汇报人：谢瓯阳

中国科学院大学 · 计算机科学与技术学院 | 2026 年 10 月 30 日

目录

1. 检索方面代码分析
2. 问题与需求
3. 优化分析
4. 方案设计
5. 实现与测试
6. 总结

PART 01 / 05

检索方面代码分析

本部分内容

核心对象 · 检索链 · 路由机制 · 三塔

Mandol 融合索引增强 · 约束下推式组合检索 · 阶段 1

汇报主线 01

1 检索方面代码分析
2 问题与需求
3 优化分析
4 方案设计
5 实现与测试
6 总结

核心对象

MemoryUnit：记忆最小单元

承载单条记忆的最小单元：保存原始数据、提供 uid，并负责从各字段中提取用于检索的文本。

SemanticGraph：记忆间关系

用有向图维护记忆间的语义关系（时序、引用、状态更新），对外提供关系维护与受约束组合检索的统一入口。

SemanticMap：唯一真相源

集中持有全部记忆数据与索引（记忆字典、FAISS、元数据索引），是取回实际记忆数据的唯一入口。

MemorySpace：记忆分组

只记录成员归属（哪些 uid 属于本空间、有哪些子空间），不持有数据。

检索链


S1 路由 _route_towers：规则表 → 塔开关 + 召回预算
S2 分发 _dispatch_towers：按开关并行跑三塔
S3 塔内召回三路并行后融合排序
S4 合并 _build_candidates：把各塔候选汇成全池，
min-max → 写入 ce_score
S5–S7 裁剪层：截尾去噪 → 跨塔仲裁，保留最优记忆 → 打包
S8–S9 输出层：统一重排 → 量化打分

路由机制：规则如何决定分发哪几座塔

路由层不参与打分，只负责决策：**开哪几座塔**

核心分发逻辑：按“问题类别”决定塔开关

- **保守策略（默认分发）**：绝大多数问题（聚合、时间、简单事实等）直接**三塔全开**（层级 + 关系 + 情景）；仅在遇到“对抗类”问题时，缩减为仅开**关系塔与情景塔**。
- **激进策略（精准分发）**：根据问题类别精准定向，避免资源浪费。
  - **聚合类** → 仅开“层级塔 + 情景塔”
  - **时间类** → 仅开“层级塔 + 关系塔”
  - **开放域 / 简单事实类** → 开启全部三塔

三塔各管一类记忆

塔决定“去哪类记忆里找”

分层塔：越往上越抽象

L0 观察：一句话、一个动作
L1 摘要：一段事件的概括
L2 洞见：跨事件的结论
三层各召回一次，再按层融合

实体关系塔：人和事怎么连

实体：人、地点、物品
关系：居住于、签租约、属于
在实体与关系两个空间里召回，回答“谁和谁有关”

情景塔：何时发生过什么

带时间戳的**事件单元**
一次经历、一段经过
回答“什么时候、先后顺序”，支持把时间注入上下文

PART 02 / 05

问题与需求

本部分内容

问题分析 · 结构性缺陷 · 需求分析

汇报主线 02

1 检索方面代码分析
2 问题与需求
3 优化分析
4 方案设计
5 实现与测试
6 总结

问题：组合查询难以实现

- **任务场景**：记忆访问同时要求语义相似、属性、时间与关系条件。
- **举例**：与内容 C 语义相近、发生在 T 之后、属于实体 E 的 1-hop 邻居、且属于空间 S。
- **现状**：三路在塔内并行召回并做分数融合，产出每塔一份统一排序；三塔取并集语义；无“必须满足”约束，属性、时间、关系只能后置过滤或全库扫描。

核心问题

如何让约束作为**硬条件**进入召回，而不是在结果上“尽量过滤”？

现有范式：并集 + 分数融合 $\Longrightarrow$ 组合查询需要：硬约束筛选

三条结构性缺陷

缺陷	后果与直接症状
并集语义	无法表达组合条件，只能靠排序“尽量”靠前
裁剪不可逆	符合条件的记忆可能被裁剪去除
裁剪层按塔优先级仲裁	图证据在高优先级塔在场时几乎无生存空间

分析

第一条是**表达能力**问题（没有约束语法），后两条是**时机**问题（约束信号来得太晚）。三者共同指向同一个缺口：召回之前，没有筛选的约束层。

目标：把约束条件提前到召回之前，而不是交给排序和裁剪去碰运气。

需求分析

需求	含义
表达能力	一条语句表达“语义 + 属性 + 时间 + 关系”的组合约束
正确性	返回结果必须硬满足全部约束，而不是“尽量”
效率	高选择性约束下避免全库扫描与无效向量比对
衔接与成本	不破坏现有检索流程与行为；维护成本可量化

PART 03 / 05

优化分析

本部分内容

选择优化位置 · 优化后的检索链

汇报主线 03

1 检索方面代码分析
2 问题与需求
3 优化分析
4 方案设计
5 实现与测试
6 总结

位置判定：选择在哪里做优化

候选位置	能力边界	分析
融合层	调权重 / RRF / 图扩展	本质是**软排序**，输出仍是打分列表；无法表达硬约束，也无法保证结果集合
裁剪层	MAD、跨塔仲裁、MMR	**时机太晚**：已在上游 top_k 之后，被截断的单元永远进不来（走查结论 1、3）
三个后端各自实现	在 BM25 / SPLADE / Cosine 内加谓词	同一套逻辑重复三份（加图与空间共五份），破坏多态，新增后端还要再写一遍
推断层 + 约束层	**路由之前求出 UID 集合，塔内下推给后端**	**采用**：唯一能在“召回之前”自动表达硬约束、且不改后端的位置

在**路由之前**加一层推断和约束（把查询变成候选集），在**三塔内部**把候选集下推为 `candidate_uids`。

检索链（优化版）


约束推断 根据查询生成约束条件
规则/空间抽取 → 判断条件
两道闸门通过 → QueryConstraints
路由（改） 按约束形态分五支
none / space_only / empty / too_broad / filtered
塔内（改） 候选集下推
candidate_uids ∩ 本塔空间 → 三路只在候选子集内打分
S4 合并归一（原样） 全池 min-max → 写入 ce_score
S5–S7 裁剪（原样） MAD 截尾 → 跨塔仲裁 → MMR 打包
S8–S9 输出（原样） 统一重排 → 量化打分
新增或改动
原样执行

PART 04 / 05

方案设计

本部分内容

建模 · 流程 · 候选集生成

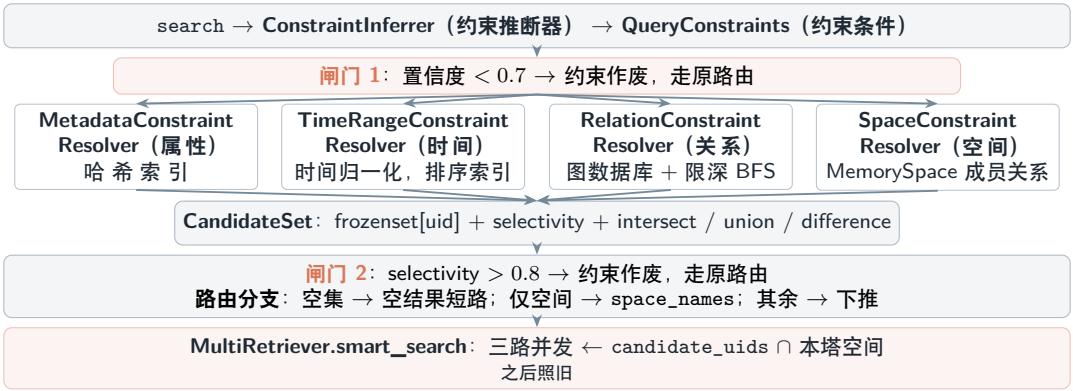
汇报主线 04

1 检索方面代码分析
2 问题与需求
3 优化分析
4 方案设计
5 实现与测试
6 总结

建模

三条建模原则

- **值对象隔离**：值对象（约束条件，候选集）不可变；下游不区分。
- **策略多态**：约束与索引均为策略族，新增类型不需要修改调度代码。
- **组合优于继承**：不继承职责过重的 `SemanticMap`，不派生索引子类；索引与调度以组合方式注入。



流程：从推断到下推的一条通路

1. **推断**：根据 query 推断约束
2. **闸门**：置信 < 0.7 或收窄 $> 0.8 \rightarrow$ 当作无约束
3. **求值**：各 Resolver $\rightarrow$ `CandidateSet`
4. **求交**：AND 语义 `intersect`；仅空间只传 `space_names`
5. **路由**：定塔开关
6. **下推**：候选集 $\cap$ 本塔空间 $\rightarrow$ `smart_search`
7. **后处理**：S4 归一 $\rightarrow$ S5–S7 裁剪 $\rightarrow$ S8–S9

两道闸门

置信度 < 0.7 不应用：宁可漏判；
收窄度 $s > 0.8$ 不应用；
都不过 $\rightarrow$ 路由与无约束时完全一致。

审计三元组

`source`：rule / declared / none
`confidence`：0–1
`dropped`：被丢弃的谓词及原因

约束推断：规则 + schema 把 query 变成约束

`constraints="auto"` → **ConstraintInferrer** → **schema 绑定** → **四类规则**
min 置信 → **两道闸门** → **QueryConstraints**

类别	触发特征	产出	置信度
时间	相对时间 / ISO 日期	`TimeRange`	0.8–0.95
标识	UUID / `#id` / 编号	`uid` 过滤	0.95
属性	< 字段 > 是/为/= 值	`eq` 过滤	0.8 / 0.9
空间	空间名 / 图谱名	`space_names`	0.80

schema 绑定：防幻觉

- 词表：已注册字段 + 采样 200 条数据
- 校验：字段类型 + 算子白名单
- 严格匹配：等值必须“见过”，相对时间必须有锚点

聚合与审计

- **min 置信**：取各项最低分（木桶效应）
- **审计丢弃**：记录 `dropped` 原因（不在字典/未挂载等）

候选集生成

属性：HashIndex

- **倒排索引**：`field` → `value` → `Set[uid]`
- **约束支持**：等于，不等，包含，不包含
- 列表/字典按键值投影
- 不可投影 → 扫描

时间：SortedIndex

- **二分查找**：排序 + `bisect` 定位区间
- **约束支持**：大于，大于等于，小于，小于等于
- **归一化**：`TimestampNormalizer` 统一格式

关系：图上 BFS

- 遍历图数据库，访问集合去重
- **方向**：后继，前驱，双向
- 限制 `max_depth` 与关系类型

PART 05 / 05

实现与测试

本部分内容

当前实现 · 测试结果

汇报主线 05

1 检索方面代码分析
2 问题与需求
3 优化分析
4 方案设计
5 实现与测试
6 总结

当前实现

阶段	具体实现	状态
P0	值对象、CandidateSet、四类 Resolver、门面	已完成
P1	字段注册与持久化、Hash/Sorted/Normalizer	已完成
P2	关系 BFS、空间约束、P2+ 关系类型倒排	已完成
P3	Planner、选择率调度、池扩容、FallbackPolicy	已完成
P4	冷层补建、容器投影、边界测试、注册表持久化	已完成
M0–M4	基线 → 规则抽取 / schema 绑定 → 两道闸门 → 路由五分支 → 塔内下推 → 分塔原始分	已完成
待办	M1c LLM 兜底 · M5 入口收敛（默认 `auto`）· M6 gold 集评测	未开始

关键代码位置与测试

`constraints/`
值对象 · Resolver · Planner · Fallback
inference.py：规则/空间 · schema · 闸门

`indexes/`
Hash / Sorted / MetadataIndex / Normalizer

`core/`
索引挂载 · 冷层补建

`tests/` **144**：约束 53 / 索引 22 / 分层 20
推断 22 / 三塔 15 / 分塔 MAD 4 / 类型 8

`examples/` A–I 九段离线基准

测试结果：执行策略的收益

16.7% → 100%
高选择性 recall@10（直连）

51.0% → 100%
宽约束 recall@10（直连）

40–87×
约束解析：索引 vs 扫描

144
全量测试通过

指标	结果	口径
关系类型查询	3.8×	基准 D 段，启用类型倒排
索引增量维护	约 7.8 μs/单元	仅索引层，不含 FAISS / 空间 / 检索器更新
索引常驻内存	约 248.5 B/单元	485.3 KB / 2000 units，峰值 485.6 KB

总结

1. **检索流程**：九段串联；路由只控开关与预算；合并用全池 min-max；三级裁剪只做减法。
2. **问题**：并集语义、裁剪不可逆、塔优先级硬碾压——约束信号来得太晚。
3. **定位**：路由太粗粒度、融合无法硬约束、裁剪太晚、后端重复；约束层落地后又暴露“入口不闭合”，合适处是推断层 + 候选集层。
4. **方案**：系统从查询推断约束 → 两道闸门 → Resolver 求值 → 路由与塔内下推 → S4–S9。
5. **实现**：P0–P4 + M0–M4、144 项测试；高选择性 recall@10 由 16.7% 到 100%，宽约束由 51.0% 到 100%。

把分散的属性、时间与图能力统一为**可组合**、**可调度**、**可审计**的约束层，并让约束有了**自动来源**。约束是塔链的**召回前置条件**。

谢谢聆听

汇报人：谢瓯阳　2026 年 10 月 30 日